

- If the code loops over the data, it's $O(n)$.
- If the code has two nested loops that each iterate over the data, it's $O(n^2)$.
- Function calls don't count as one step but rather the total steps of the code inside the function. See "Big O Order of Common Function Calls" on page 242.
- If the code has a DIVIDE AND CONQUER step that repeatedly halves the data, it's $O(\log n)$.
- If the code has a DIVIDE AND CONQUER step that is done once per item in the data, it's an $O(n \log n)$.
- If the code goes through every possible combination of values in the n data, it's $O(2^n)$, or some other exponential order.
- If the code goes through every possible permutation (that is, ordering) of the values in the data, it's $O(n!)$.
- If the code involves sorting the data, it will be at least $O(n \log n)$.

These rules are good starting points. But they're no substitute for actual big O analysis. Keep in mind that big O order isn't the final judgment on whether code is slow, fast, or efficient. Consider the following `waitAnHour()` function:

```
import time
def waitAnHour():
    time.sleep(3600)
```

Technically, the `waitAnHour()` function is $O(1)$ constant time. We think of constant time code as fast, yet its runtime is one hour! Does that make this code inefficient? No: it's hard to see how you could have programmed a `waitAnHour()` function that runs faster than, well, one hour.

Big O isn't a replacement for profiling your code. The point of big O notation is to give you insights as to how the code will perform under increasing amounts of input data.